

Methods for resolving list performance problems

1. Pagination

Pagination enables you to make data in pages rather than all at once. This way, you can effectively monitor the amount of data shown on the website without undue strain on the DOM tree.

Although most React UI libraries have a pagination feature, if you're looking for a fast way to enforce pagination without installing a UI library, you might want to check out `react-paginate`. The library generates a pagination component that accepts certain props and aids in data navigation.

Run the following code in your terminal to install the library. Please note, Part 1 and Part 2 of the Pagination implementation code are to be written together.

```
import React, { useState, useEffect } from 'react';
import ReactPaginate from 'react-paginate';
import faker from 'faker'
import './App.css';

function App() {
  const [pagination, setPagination] = useState({
    data: new Array(1000).fill().map((value, index) => ({
      id: index,
      title: faker.lorem.words(5),
      body: faker.lorem.sentences(8)
    })),
    offset: 0,
    numberPerPage: 10,
    pageCount: 0,
    currentData: []
  });
```

Pagination implementation code (Part 1)

Copyright: Log rocket

Following installation, you should configure your app component to paginate the data rather than rendering it all at once.

In this case, we're storing the pagination information in our paginating state. Rather than making the whole data set at once, we make only the present data, which we obtain by splitting the key data set according to the current offset and the number of records to be shown on the page.

As a prop, the `ReactPaginate` function accepts an event handler that is invoked whenever the page shifts. When the page loads, the event controller